

QUESTION 1:

Steps:

My steps heuristic is the number of letters that are different from the goal word.
 $h(n)$ =number of mismatched letters

Since one move can only change one letter, every mismatched position has to be changed at least once. Each move costs 1, so this is the minimum possible remaining step cost. It cannot overestimate because the actual solution may need extra moves, but it cannot need fewer moves than the number of mismatched letters

Scrabble:

For Scrabble, I add the value of each goal letter that is not currently in the correct position.

$h(n)=\sum \text{ScrabbleValue}(\text{goal letter})$ for every mismatched position.

Each mismatched position must eventually receive its correct goal letter, so the Scrabble cost of that letter must be paid at least once. The heuristic does not include any extra letters that may need to be inserted along the path, so it can underestimate the cost, but it cannot overestimate it.

Frequency:

For frequency cost, I use the number of mismatched letters.

$h(n)$ =number of mismatched letters

Every move costs $1+\text{rarity}(w)$, and rarity is nonnegative, so each move costs at least 1. Since every mismatched position needs at least one move to fix, the number of mismatches is a minimum possible remaining frequency cost. It may underestimate the real cost because it ignores rarity values and extra moves, but it cannot overestimate it.

QUESTION 2:

Word pair	cost measure	UCS cost	UCS nodes expanded	A* cost	A* expanded	reduction
WHY → ASK	Steps	5	773	5	16	97.98%
WHY → ASK	Scrabble	9	765	9	13	98.30%
WHY → ASK	Frequency	38.8554	808	38.8554	755	6.56%
BEAR → DUCK	Steps	4	691	4	6	99.13%
Bear → duck	Scrabble	11	2981	11	6	99.80%
Bear → duck	Frequency	38.2402	1084	38.2402	720	33.58%

1. **Did UCS and A* always return the same optimal cost? If not, identify and investigate the discrepancy.:** UCS and A* gave the same optimal cost in all six tests. For why → ask, both algorithms got a cost of 5 for steps, 9 for Scrabble, and 38.8554 for frequency. For bear → duck, both got 4 for steps, 11 for Scrabble, and 38.2402 for frequency. Since the costs matched in every test, there was no difference that needed to be investigated.
2. **For each word pair, under which cost measure did A* produce the largest percentage reduction in expanded**
Nodes?: For both word pairs, A* had its biggest improvement when using the Scrabble cost. In the why → ask test, the number of expanded nodes went from 765 with UCS to only 13 with A*, which is about a 98.30% decrease. The same pattern happened for bear → duck, where the expanded nodes decreased from 2981 to 6, giving about a 99.80% reduction.
3. **For each word pair, under which cost measure was the heuristic least informative?:** The frequency heuristic helped the least for both word pairs. For → Ask, it only reduced the number of expanded nodes by about 6.56%, and for bear → duck, the reduction was about 33.58%. Compared to steps and Scrabble, those reductions are a lot smaller.
4. **Explain the differences using the definitions of your heuristics, not runtime alone:** The steps heuristic counts how many letters are different from the goal, so it gives a useful estimate of how many moves are still needed. The Scrabble heuristic gives even more information because it also includes the Scrabble values of the goal letters that still need to be inserted. The frequency heuristic is less informative because it only counts mismatched letters and does not use the rarity values at all. Since frequency costs can be much larger than 1, ignoring rarity makes the estimate much lower than the real remaining cost.

QUESTION 3:

Steps:

Yes, the cost should be the same in both directions. Every move costs 1, and if you can change one word into another, you can also reverse that change. So if it takes a certain number of moves to go from one word to another, reversing that path takes the same number of moves.

Scrabble:

No, the cost does not have to be the same. The move itself is reversible, but the cost depends on the letter that gets inserted. Going one direction may insert cheaper letters than going the other direction.

For ex:

Why - > ask = 9

But

Ask -> why = 14

So even though the actions can be reversed, the total Scrabble cost can be different.

Frequency

No, the cost can also be different in each direction. Frequency cost depends on the rarity of the word you move into. When the path is reversed, you enter a different set of words in a different order, so the total cost can change.

Ex

Why -> ask = 38.86

While

Ask -> why = 37.76

So the main difference is that the actions are reversible, but the costs are only symmetric for the steps measured.

Question 4

In DOGCAT, the initial and goal states are lowercase words of the same length—three or four letters—and every legal state must appear in the supplied dictionary. A legal action changes exactly one letter. Under the frequency objective, entering word w costs $1 + \text{rarity}(w)$, where the nonnegative rarity value is stored in the dictionary.

Show more ▾

Worked for 40s

A good rarity-aware heuristic is a **one-step lookahead heuristic**:

$$h_R(n) = \begin{cases} 0, & n = \text{goal} \\ \min_{s \in \text{Succ}(n)} (1 + \text{rarity}(s) + h_0(s)), & n \neq \text{goal} \end{cases}$$

where $\text{Succ}(n)$ is the set of legal words you can reach from n by changing exactly one letter.

Why this works

For every possible next word s :

- $1 + \text{rarity}(s)$ is the actual cost of entering that next word

version not said

2. Pseudocode

if current word is the goal:
 return 0

lowest = infinity

for each legal next word:
 estimate = $1 + \text{rarity}(\text{next word}) + h_0(\text{next word})$

 if estimate < lowest:
 lowest = estimate

return lowest

3.

Yes, it is admissible.

The first move from the current word has to happen no matter what.

That move costs: **$1 + \text{rarity}(\text{next word})$**

After that, h_0 estimates how many letters are still wrong.

Since each move can only change one letter, h_0 does not overestimate the number of moves still needed.

Also, every frequency move costs at least 1. And so Because of that, this estimate:

$1 + \text{rarity}(\text{next word}) + h_0(\text{next word})$

does not go above the real cost of reaching the goal through that next word.

The heuristic chooses the smallest value from all legal next words, so it still does not overestimate the true best path. It is also at least as large as h_0 .

One move can fix at most one wrong letter, so after one move:

$$h_0(\text{next word}) \geq h_0(\text{current word}) - 1$$

Since rarity is never negative:

$$1 + \text{rarity}(\text{next word}) + h_0(\text{next word}) \geq h_0(\text{current word})$$

So the new heuristic is never smaller than the original one.

4.

No, it does not need to be changed.

The proposed heuristic is already admissible.

It uses:

- the real cost of the next move
- plus the normal mismatch heuristic after that move

Both parts are safe lower bounds, so the heuristic does not overestimate. And Because of that, I would keep the heuristic as it is.

5.

Word pair	Heuristic	Solution Cost	Nodes Expanded
why->ask	h0	38.8554	755
why->ask	hR	38.8554	615
bear-> duck	h0	38.2402	720
bear-> duck	hR	38.2402	418

It is shown for the why->ask the new heuristic reduced the expanded nodes from **755 to 615**.

That is about an **18.54% reduction**.

And for the bear -> duck the expanded nodes went from **720 to 418**.

That is about a **41.94% reduction**.

The solution cost stayed the same in both cases, which means the new heuristic still found the optimal solution.

So based on these tests, the rarity-aware heuristic was more useful because it caused A* to expand fewer nodes while keeping the same optimal/better cost.